

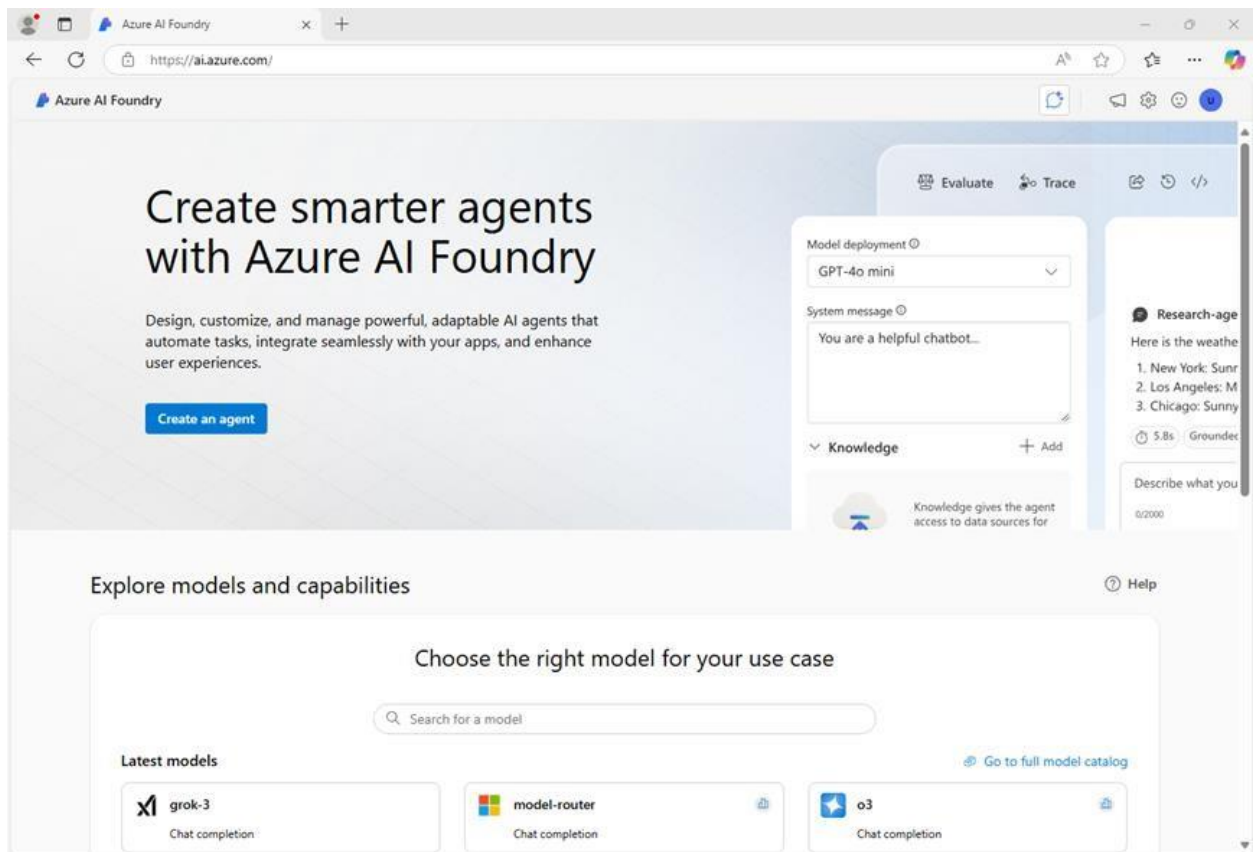
Develop an Azure AI chat agent with the Microsoft Agent Framework SDK

In this exercise, you'll use Azure AI Agent Service and Microsoft Agent Framework to create an AI agent that processes expense claims.

Deploy a model in a Microsoft Foundry project

Let's start by deploying a model in a Foundry project.

1. In a web browser, open the Foundry portal at <https://ai.azure.com> and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the **Foundry** logo at the top left to navigate to the home page, which looks similar to the following image (close the **Help** pane if it's open):

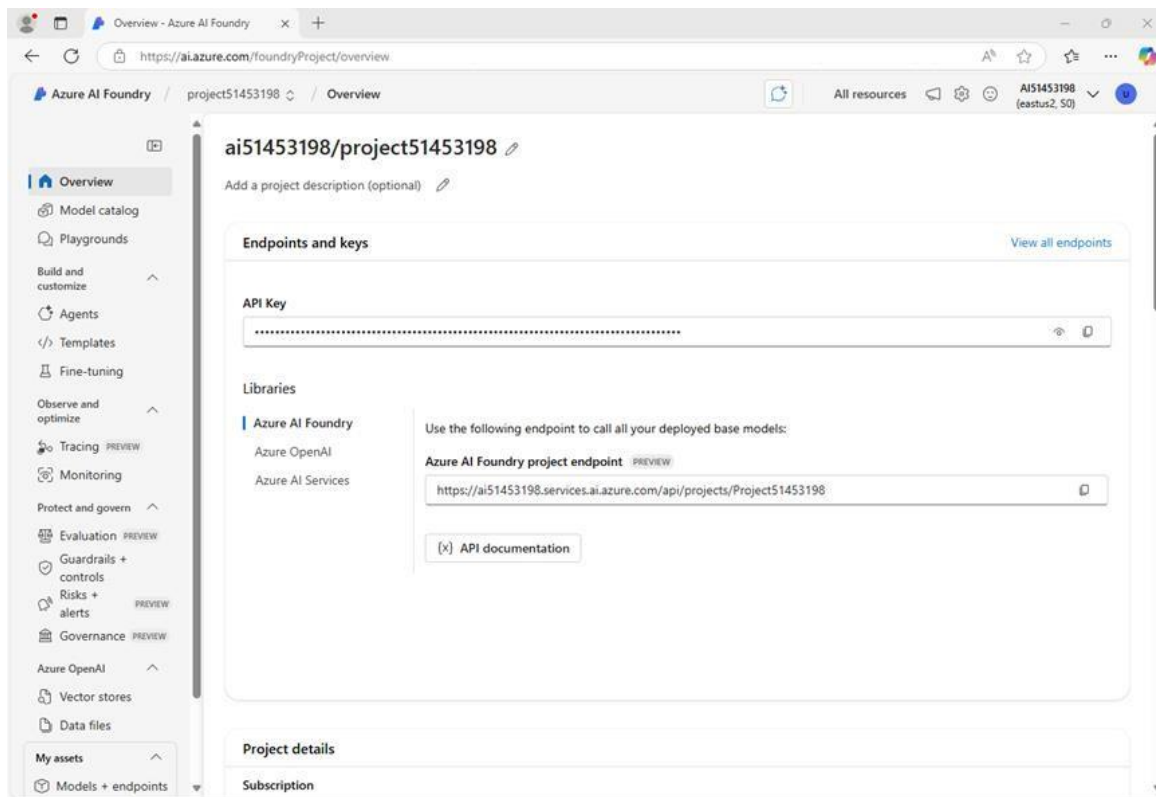


Important: Make sure the **New Foundry** toggle is *Off* for this lab.

2. On the home page, in the **Explore models and capabilities** section, search for the gpt-4o model; which we'll use in our project.
3. In the search results, select the **gpt-4o** model to see its details, and then at the top of the page for the model, select **Use this model**.
4. When prompted to create a project, enter a valid name for your project and expand **Advanced options**.
5. Confirm the following settings for your project:
 - a. **Foundry resource:** *A valid name for your Foundry resource*
 - b. **Subscription:** *Your Azure subscription*
 - c. **Resource group:** *Create or select a resource group*
 - d. **Region:** *Select any **AI Foundry recommended****

* Some Azure AI resources are constrained by regional model quotas. In the event of a quota limit being exceeded later in the exercise, there's a possibility you may need to create another resource in a different region.

6. Select **Create** and wait for your project, including the gpt-4 model deployment you selected, to be created.
7. When your project is created, the chat playground will be opened automatically.
8. In the **Setup** pane, note the name of your model deployment; which should be **gpt-4o**. You can confirm this by viewing the deployment in the **Models and endpoints** page (just open that page in the navigation pane on the left).
9. In the navigation pane on the left, select **Overview** to see the main page for your project; which looks like this:



Create an agent client app

Now you're ready to create a client app that defines an agent and a custom function. Some code has been provided for you in a GitHub repository.

Prepare the environment

1. Open a new browser tab (keeping the Foundry portal open in the existing tab). Then in the new tab, browse to the [Azure portal](https://portal.azure.com) at <https://portal.azure.com>; signing in with your Azure credentials if prompted.

Close any welcome notifications to see the Azure portal home page.

2. Use the **[>]** button to the right of the search bar at the top of the page to create a new Cloud Shell in the Azure portal, selecting a **PowerShell** environment with no storage in your subscription.

The cloud shell provides a command-line interface in a pane at the bottom of the Azure portal. You can resize or maximize this pane to make it easier to work in.

Note: If you have previously created a cloud shell that uses a *Bash* environment, switch it to **PowerShell**.

3. In the cloud shell toolbar, in the **Settings** menu, select **Go to Classic version** (this is required to use the code editor).

Ensure you've switched to the classic version of the cloud shell before continuing.

4. In the cloud shell pane, enter the following commands to clone the GitHub repo containing the code files for this exercise (type the command, or copy it to the clipboard and then right-click in the command line and paste as plain text):

```
rm -r ai-agents -f
git clone https://github.com/KiranAvulasetty/ai-agents ai-agents
```

Tip: As you enter commands into the cloudshell, the output may take up a large amount of the screen buffer and the cursor on the current line may be obscured. You can clear the screen by entering the `cls` command to make it easier to focus on each task.

5. When the repo has been cloned, enter the following command to change the working directory to the folder containing the code files and list them all.

```
cd ai-agents/Labfiles/04-agent-framework/python
ls -a -l
```

The provided files include application code a file for configuration settings, and a file containing expenses data.

Configure the application settings

1. In the cloud shell command-line pane, enter the following command to install the libraries you'll use:

```
python -m venv labenv
./labenv/bin/Activate.ps1
pip install -r requirements.txt
```

2. Enter the following command to edit the configuration file that has been provided:

```
code .env
```

The file is opened in a code editor.

3. In the code file, replace the **your_project_endpoint** placeholder with the endpoint for your project (copied from the project **Overview** page in the Foundry portal), and the **your_model_deployment** placeholder with the name you assigned to your gpt-4o model deployment.
4. After you've replaced the placeholders, use the **CTRL+S** command to save your changes and then use the **CTRL+Q** command to close the code editor while keeping the cloud shell command line open.

Write code for an agent app

1. Open the **agent-framework.py** file in the code editor.
2. Review the code in the file. It contains:
 - Some **import** statements to add references to commonly used namespaces
 - A *main* function that loads a file containing expenses data, asks the user for instructions, and then calls...
 - A **process_expenses_data** function in which the code to create and use your agent must be added
3. At the top of the file, after the existing **import** statement, find the comment **Add references**, and add the following code to reference the namespaces in the libraries you'll need to implement your agent:

```
code
```

```
# Add references
from agent_framework import tool, Agent
from agent_framework.azure import AzureOpenAIResponsesClient
from azure.identity import AzureCliCredential
from azure.identity.aio import AzureCliCredential
from pydantic import Field
from typing import Annotated
```

4. Near the bottom of the file, find the comment **Create a tool function for the email functionality**, and add the following code to define a function that your agent will use to send email (tools are a way to add custom functionality to agents)

code

```
# Create a tool function for the email functionality
@tool(approval_mode="never_require")
def submit_claim(
    to: Annotated[str, Field(description="Who to send the email to")],
    subject: Annotated[str, Field(description="The subject of the email.")],
    body: Annotated[str, Field(description="The text body of the email.")]:
    print("\nTo:", to)
    print("Subject:", subject)
    print(body, "\n")
```

Note: The function *simulates* sending an email by printing it to the console. In a real application, you'd use an SMTP service or similar to actually send the email!

5. Back up above the **send_email** code, in the **process_expenses_data** function, find the comment **Create a client and initialize an agent with the tool and instructions**, and add the following code:

(Be sure to maintain the indentation level)

code

```
# Create a client and initialize an agent with the tool and instructions
credential = AzureCliCredential()
async with (
    Agent(
        client=AzureOpenAIResponsesClient(
            credential=credential,
            deployment_name=os.getenv("MODEL_DEPLOYMENT_NAME"),
            project_endpoint=os.getenv("PROJECT_ENDPOINT"),
        ),
        instructions="""You are an AI assistant for expense claim submission.
        At the user's request, create an expense claim and use the plug-
        in function to send an email to expenses@contoso.com with the subject 'Expense
        Claim`and a body that contains itemized expenses with a total.
        Then confirm to the user that you've done so. Don't ask for any
        more information from the user, just use the data provided to create the email.""",
        tools=[submit_claim],
    ) as agent,
):
```

Note that the **AzureCliCredential** object will allow your code to authenticate to your Azure account. The **AzureOpenAIResponsesClient** object includes the Foundry project settings from the .env configuration. The **Agent** object is initialized with the client, instructions for the agent, and the tool function you defined to send emails.

- Find the comment **Use the agent to process the expenses data**, and add the following code to create a thread for your agent to run on, and then invoke it with a chat message.

(Be sure to maintain the indentation level):

code

```
# Use the agent to process the expenses data
try:
    # Add the input prompt to a list of messages to be submitted
    prompt_messages = [f"{prompt}: {expenses_data}"]
    # Invoke the agent for the specified thread with the messages
    response = await agent.run(prompt_messages)
    # Display the response
    print(f"\n# Agent:\n{response}")
except Exception as e:
    # Something went wrong
    print (e)
```

- Review that the completed code for your agent, using the comments to help you understand what each block of code does, and then save your code changes (**CTRL+S**).

Run the app

- In the integrated terminal, enter the following command to run the application:

code

```
python agent-framework.py
```

- When asked what to do with the expenses data, enter the following prompt:

code

```
Submit an expense claim
```

- When the application has finished, review the output. The agent should have composed an email for an expenses claim based on the data that was provided.

4. When you're finished, enter `deactivate` in the terminal to exit the Python virtual environment.

Summary

In this exercise, you used the Microsoft Agent Framework SDK to create an agent with a custom tool.